

WhisperChat (NBchat) 项目总结报告

项目名称：WhisperChat / NBchat — Jakarta EE 实时在线聊天系统

版本：v1_2026_5_30

技术栈：Jakarta EE 10 + MySQL 8 + WebRTC + WebSocket

部署地址：<https://nbchatroom.cloud>

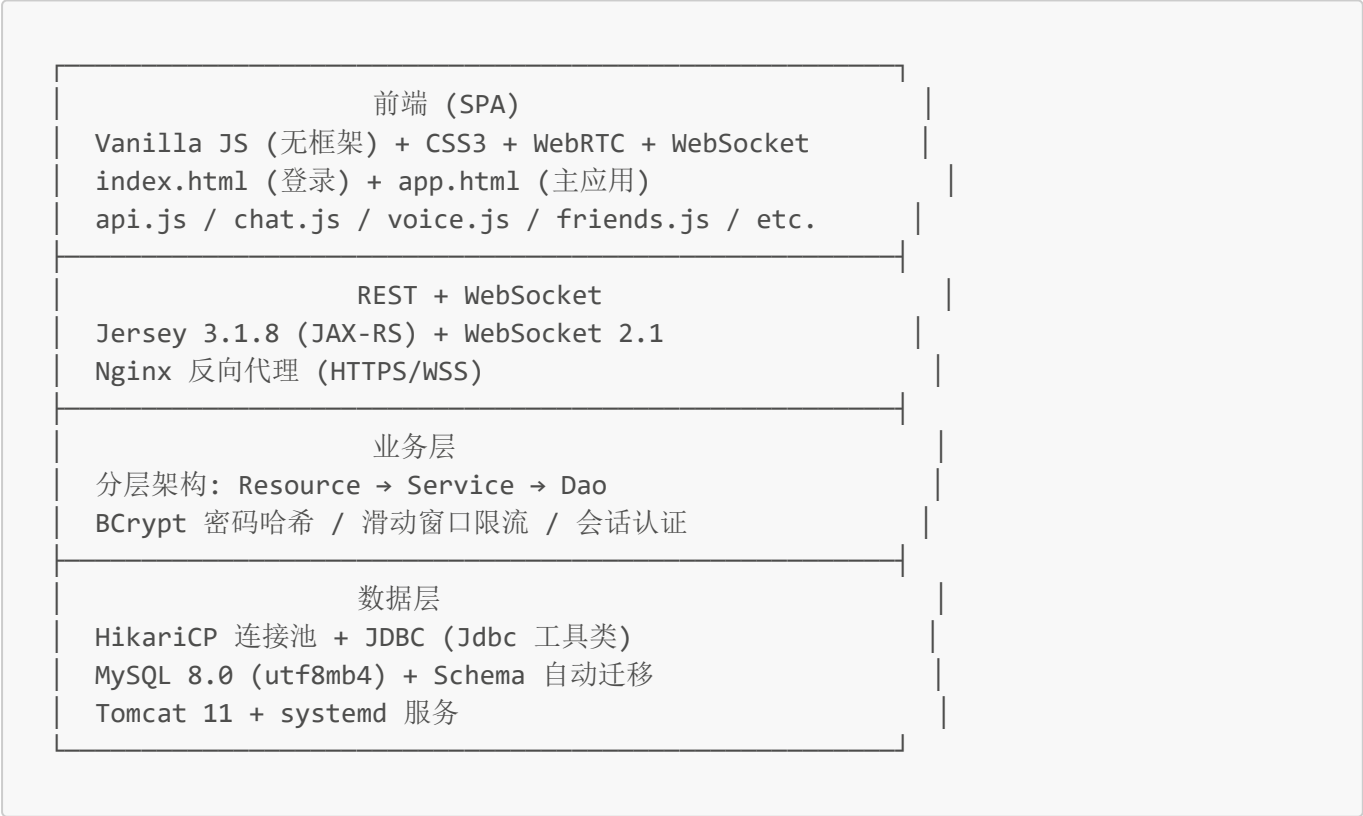
代码仓库：https://github.com/Jiduandeyang/v1_2026_5_30_NBchat

一、项目概述

WhisperChat 是一个基于 Jakarta EE 10 的全栈实时在线聊天系统，支持文本/图片/语音消息、群组管理、朋友圈动态、语音视频通话、AI 助手、后台管理等完整功能。项目采用纯 Java 标准技术栈，无需 Spring 等第三方框架，前端使用原生 JavaScript（无框架依赖），展现了 Jakarta EE 规范的完整运用能力。

- **总代码量：** 约 11,321 行（含 Java 后端 ~6,500 行、JavaScript 前端 ~3,500 行、SQL ~344 行、CSS ~2,800 行）
- **Java 源文件：** 90 个（含测试 28 个）
- **数据库表：** 23 张
- **REST API 端点：** 60+ 个
- **WebSocket 端点：** 2 个（Chat + Voice）

二、技术架构



核心代码位置：

- 后端入口：`src/main/java/com/example/chat/ChatApplication.java`
- REST 配置：通过 `@ApplicationPath("/api")` 注册所有资源类

- 前端入口：`src/main/webapp/index.html` (登录) + `src/main/webapp/app.html` (主应用)
- 数据库 Schema：`src/main/resources/schema.sql`
- Schema 迁移：`src/main/java/com/example/chat/config/SchemaMigrator.java`
- 数据库连接池：`src/main/java/com/example/chat/config/Database.java`
- 配置管理：`src/main/java/com/example/chat/config/AppConfig.java`

三、模块全景

模块	后端文件	前端文件	功能概述
Auth (认证)	AuthResource/Service/Dao + EmailCode + QQMail	auth.js + index.html	注册、登录、密码重置、QQ 邮箱验证码
Chat (聊天)	ChatResource/Service/Dao + 15 DTO	chat.js (1855 行)	私聊/群聊/消息/回复/撤回/Reaction/投票/胶囊/阅后即焚
Voice (语音视频)	VoiceResource/Service/Dao + Notifier	voice.js + call-*.js	WebRTC 语音/视频通话, perfect-negotiation
Friend (好友)	FriendResource/Service/Dao	friends.js (310 行)	好友添加/分组/亲密好友/请求管理
Moment (朋友圈)	MomentResource/Service/Dao + VisibilityPolicy	moments.js	动态发布/点赞/评论/可见性控制
Media (媒体)	MediaResource/Service/Dao + UploadStaticServlet	chat.js (上传部分)	图片/视频/音频上传、类型校验、Range 下载
AI (AI助手)	AiAssistantService + DeepSeekClient	chat.js (触发逻辑)	@AI助手 群聊AI回复
Admin (后台管理)	AdminResource/Service/Dao + 5 Row/Page类	admin.js (214 行)	仪表盘/用户管理/群管理/动态管理/审计日志
WebSocket	ChatEndpoint + VoiceEndpoint + SocketRegistry	chat.js + voice.js	实时消息推送、WebRTC 信令中继
User (用户)	UserResource/Service/Dao	profile.js	个人资料/用户搜索/公开主页

四、项目亮点

1. 纯 Jakarta EE 标准技术栈

- 位置：整个 `src/main/java/com/example/chat/` 目录
- 零 Spring 依赖，使用 Jersey (JAX-RS)、Tomcat WebSocket、Servlet API
- 手动依赖注入、函数式 JDBC 封装、Record 不可变数据模型
- 展示了对 Jakarta EE 规范的深入理解

2. WebRTC 语音/视频通话 — W3C Perfect Negotiation 模式

- **位置:** `src/main/webapp/assets/js/voice.js` (273行) + `src/main/java/com/example/chat/voice/`
- 完整实现 W3C perfect-negotiation 协商模式, 解决"抢先 offer"竞技问题
- 主叫方在收到被叫方 offer 后才添加本地音轨, 确保双方双向音频
- 支持 voice call + video call 两种模式, WebSocket 信令 + REST 轮询双重保障
- ICE 服务器可配 (STUN/TURN), call_mode 数据库区分

3. WebSocket + REST 双重保障

- **位置:** `src/main/webapp/assets/js/chat.js` (sendMessage 函数, ~1113行) + `src/main/java/com/example/chat/websocket/ChatEndpoint.java`
- 消息优先走 WebSocket 实时通道, 失败自动降级到 REST
- Reaction、Recall、Group Invitation 三种事件通过 WebSocket 事件分发

4. 创新消息类型

- **位置:** `src/main/java/com/example/chat/chat/SendMessageRequest.java` (type 字段)
- **时光胶囊 (TIME_CAPSULE):** 定时解锁消息, 到期前内容加密显示, `src/main/webapp/assets/js/chat.js` renderTimeCapsuleMessage (~644行)
- **阅后即焚 (BURN):** Canvas 2D 刮开效果, `src/main/webapp/assets/js/chat.js` renderBurnMessage (~560行) + wireBurnCanvases (~860行)
- **投票 (POLL):** !poll 语法解析, `src/main/webapp/assets/js/chat.js` renderPoll (~1684行)
- **语音消息 (VOICE):** MediaRecorder API, `src/main/webapp/assets/js/chat.js` startVoiceRecording (~1225行)

5. 精细的群组管理

- **位置:** `src/main/java/com/example/chat/chat/ChatService.java` (572行)
- 三级角色体系 (OWNER/ADMIN/MEMBER) + requireOwner/requireManager/requireMember 权限检查
- 群公告 (announcement)、群邀请策略 (亲密好友直接入群)、成员角色管理
- 群背景自定义 (4种预设 + 自定义图片)、群备注、静音

6. 朋友圈可见性系统

- **位置:** `src/main/java/com/example/chat/moment/VisibilityPolicy.java` (61行) + `src/main/java/com/example/chat/moment/MomentDao.java` feed SQL
- 四种可见性模式: ALL_FRIENDS / ONLY_SELF / SELECTED / EXCLUDE
- 可见性规则完全在 SQL 层面实现, 性能高效
- 支持按用户和按分组两种维度筛选

7. AI 群聊助手

- **位置:** `src/main/java/com/example/chat/ai/AiAssistantService.java` + `DeepSeekClient.java`
- @AI助手名称 触发, 将最近30条群聊上下文发送给 DeepSeek
- DeepSeek API (OpenAI 兼容协议), 可更换模型
- 仅群聊可用, 且需明确 @提及

8. 完整后台管理系统

- **位置:** `src/main/java/com/example/chat/admin/`
- 仪表盘 (8项实时统计)、用户管理 (搜索/角色/禁用)、群管理 (查看/解散)、动态管理 (查看/删除)
- 审计日志: 所有管理员操作自动记录 (admin_id, action, target_type, target_id, detail)
- 前端5个Tab页, 分页浏览

9. Schema 自动迁移

- **位置:** `src/main/java/com/example/chat/config/SchemaMigrator.java` (185行)
- 启动时自动检测并创建缺失的表和列, 无需手动执行 DDL
- 每个迁移独立、幂等 (hasTable/hasColumn 检查)
- 同时清理过期语音通话记录

10. BCrypt 密码哈希 + 滑动窗口限流

- **位置:** `src/main/java/com/example/chat/common/PasswordHasher.java` (12轮) + `src/main/java/com/example/chat/common/RateLimiter.java` (滑动窗口)
- 登录: 8次/5分钟, 邮箱验证码: 3次/10分钟

11. Cursor 分页 + 软删除

- **位置:** `src/main/java/com/example/chat/chat/ChatDao.java` history 查询 (beforeId 游标)
- 聊天记录使用 beforeId 游标分页而非 OFFSET, 避免大偏移量性能问题
- 所有删除操作使用软删除 (active/deleted/hidden 标记)

12. Canvas 粒子特效 + 刮开效果

- **位置:** `src/main/webapp/assets/js/chat.js` animateFxParticles (~194行) + wireBurnCanvases (~860行)
- 消息发送时粒子爆发动画
- 阅后即焚消息 Canvas 涂层刮开效果

五、数据库设计总览

共 23 张表, 核心 E-R 关系:

```
users (用户)
├── friend_groups (好友分组)
├── friendships (好友关系)
├── friend_requests (好友申请)
├── media_files (上传文件)
├── email_verification_codes (邮箱验证码)
├── voice_call_sessions (通话记录)
└── admin_audit_logs (审计日志)

conversations (会话)
├── conversation_members (会话成员 + 个人设置)
├── conversation_reads (已读追踪)
└── messages (消息)
    └── message_reactions (消息Reaction)
```



详细 Schema 见 [src/main/resources/schema.sql](#) (344行)。

六、部署架构



部署脚本位于项目根目录的 [deploy.bat](#)，通过 SSH 上传 WAR 并重启 Tomcat。

七、测试覆盖

28 个测试文件覆盖关键契约：

类别	测试文件
Auth	AuthEmailContractTest, AuthRegistrationPolicyTest, EmailCodeCooldownTest
Chat	ChatGroupManagementContractTest, CreativeChatFeaturesContractTest, GroupSettingsContractTest, GroupInvitePolicyTest, HistoryPageRequestTest
Voice	RealtimeCallModuleContractTest, IceServerConfigTest
AI	AiAssistantServiceTest, DeepSeekClientTest

类别	测试文件
Media	MediaContentTypePolicyTest, UploadSizePolicyTest, UploadKindTest
Web	WebAssetContractTest, CompletionContractTest
Config	AppConfigTest, DatabaseConfigTest, SeedDataTest
Common	PasswordHasherTest, RateLimiterTest, ValidationTest

测试代码位置：[src/test/java/com/example/chat/](#)

八、项目文件结构

v1_2026_5_30/	
├─ pom.xml	# Maven 配置 (Jakarta EE 10)
├─ deploy.bat	# 部署脚本
├─ src/	
│ └─ main/	
│ └─ java/com/example/chat/	
│ ├─ ChatApplication.java	# JAX-RS Application
│ ├─ admin/ (11 files)	# 后台管理模块
│ ├─ ai/ (2 files)	# AI助手模块
│ ├─ auth/ (8 files)	# 认证模块
│ ├─ chat/ (15 files)	# 聊天核心模块
│ ├─ common/ (9 files)	# 公共工具
│ ├─ config/ (4 files)	# 配置/数据库/Schema迁移
│ ├─ friend/ (7 files)	# 好友模块
│ ├─ media/ (8 files)	# 媒体上传模块
│ ├─ model/ (16 files)	# 数据模型 (Java Record)
│ ├─ moment/ (6 files)	# 朋友圈模块
│ ├─ user/ (4 files)	# 用户模块
│ ├─ voice/ (7 files)	# 语音/视频模块
│ └─ websocket/ (4 files)	# WebSocket模块
│ └─ resources/	
│ └─ schema.sql	# 数据库建表+种子数据
└─ webapp/	
├─ index.html	# 登录页
├─ app.html	# 主应用 (SPA)
├─ favicon.svg	# 网站图标
└─ assets/	
└─ css/dashboard.css	# 主样式表 (2874行)
└─ js/	
├─ api.js	# API客户端
├─ app.js	# 应用入口/状态管理
├─ auth.js	# 认证逻辑
├─ chat.js	# 聊天核心 (1855行)
├─ friends.js	# 好友管理 (310行)
├─ moments.js	# 朋友圈
├─ profile.js	# 个人资料
├─ voice.js	# 语音视频入口 (273行)
└─ admin.js	# 后台管理 (214行)

```
|
|
|   └─ call-api.js           # 通话REST API
|   └─ call-signaling.js    # WebRTC信令
|   └─ call-rtc.js          # WebRTC PeerConnection
|   └─ call-ui.js           # 通话UI
└─ test/java/com/example/chat/ # 28个测试文件
docs/                          # 项目文档
└─ PROJECT_SUMMARY_REPORT.md  # 本文档
└─ MODULE_REPORT_AUTH.md
└─ MODULE_REPORT_CHAT.md
└─ MODULE_REPORT_VOICE.md
└─ MODULE_REPORT_FRIEND.md
└─ MODULE_REPORT_MOMENT.md
└─ MODULE_REPORT_MEDIA.md
└─ MODULE_REPORT_AI.md
└─ MODULE_REPORT_ADMIN.md
└─ MODULE_REPORT_WEBSOCKET.md
```